

Embeddings: Words as Vectors

How numbers start to carry meaning · video 2:28

This lesson has one big idea: every token gets a **vector**, a list of numbers that places it as a **point in space**, and after training, **where a token sits carries meaning**. Similar tokens end up close together, closeness can be measured with the dot product, and even directions can mean something.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. IDs are labels, vectors are points	▶ 0:08-0:30	3	☐
2. The embedding matrix: one row per token	▶ 0:30-0:58	4	☐
3. Similar tokens end up close together	▶ 0:58-1:12	3	☐
4. Measuring closeness: dot product and cosine	▶ 1:12-1:31	4	☐
5. Directions carry meaning	▶ 1:31-1:45	3	☐
6. Learned, not designed	▶ 1:45-1:56	3	☐
7. Embeddings in code	▶ 1:56-2:25	3	☐

Before you start: this lesson builds on episode 2 (tokens and token IDs). The maths is vectors and the dot product: Foundations F01 (Vectors) and F02 (The Dot Product) cover it. Concept 7 reads a few lines of NumPy; F11 (NumPy in Three Minutes) helps.

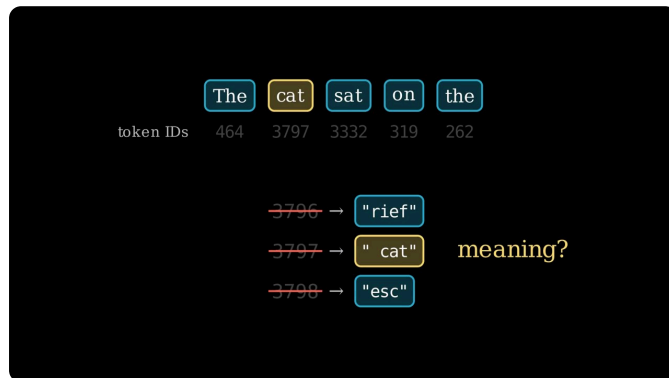
CONCEPT 1 OF 7

IDs are labels, vectors are points

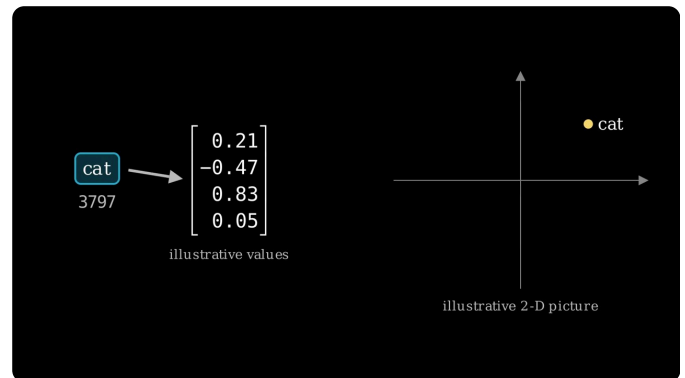
▶ Video 0:08-0:30

After tokenizing, “The cat sat on the” is [464, 3797, 3332, 319, 262]. But an ID is **just a label**. Token 3797 is `_cat`, while its neighbours 3796 and 3798 are `rief` and `esc`. The numbers say nothing about meaning.

The fix: give every token a **vector**, a list of numbers, like coordinates. Then each token becomes a **point in space**, and where it sits can carry meaning.



▶ 0:21 Token 3797 is `_cat`, and its neighbours are `rief` and `esc`: the ID numbers mean nothing.



▶ 0:30 Give each token a vector, like coordinates, and it becomes a point in space.

KEY IDEA

An ID only says **which** token it is. A token's **embedding** is its vector: a point in space whose position can say something about **what the token means**.

Check 1: IDs are labels, vectors are points

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 TRUE OR FALSE

“IDs 3796 and 3797 are neighbours, so `rief` and `_cat` must have similar meanings.”

True ☐ False ☐

1.2 MULTIPLE CHOICE

A token's embedding is...

- A. its ID written in binary
- B. a list of numbers (a vector) that places it as a point in space
- C. how often it appears in the training text
- D. the list of its letters

1.3 MULTIPLE CHOICE

Why not feed the raw ID numbers into the network as they are?

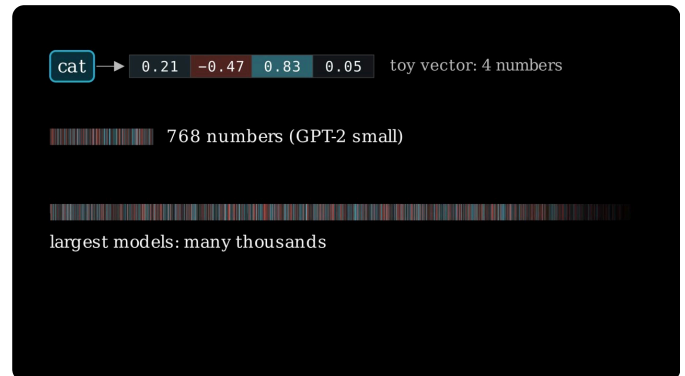
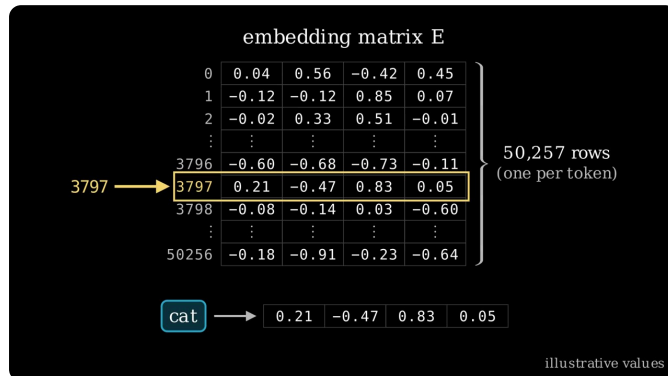
- A. They are too big to store
- B. The network would treat 3797 and 3798 as almost equal, though `_cat` and `esc` are unrelated
- C. IDs change every time you tokenize
- D. There are not enough IDs for every token

CONCEPT 2 OF 7

The embedding matrix: one row per token

▶ Video 0:30-0:58

All the vectors are stored in one big table, the **embedding matrix** E , with **one row for every token** in the vocabulary: 50,257 rows for GPT-2. Embedding a token is just a **lookup**: token 3797 grabs row 3797.



▶ 0:44 Embedding is a lookup: token 3797 grabs row 3797 of the embedding matrix (illustrative values).

▶ 0:55 Real vectors are long: 768 numbers in GPT-2 small, many thousands in the largest models.

Real vectors are long. The smallest GPT-2 uses **768 numbers per token**, and the largest models use many thousands. Pictures show just two or three, so we can see them.

KEY IDEA

Embedding = **row lookup**: token ID $i \rightarrow$ row i of E . E has one row per token in the vocabulary and one column per number in the vector (768 for GPT-2 small).

Check 2: The embedding matrix: one row per token

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

GPT-2 small's embedding matrix has 50,257 rows and 768 columns. How many numbers does it hold?

2.2 MULTIPLE CHOICE

To embed $\text{_\textit{on}}$ (ID 319), the model...

- A. multiplies 319 by every row of E
- B. takes row 319 of E
- C. searches E for the row closest to 319
- D. averages all the rows of E

2.3 SHORT ANSWER

A toy model has a vocabulary of 1,000 tokens and uses 64 numbers per token. What is the shape of its embedding matrix (rows $\times$ columns)?

2.4 TRUE OR FALSE

"The same token, used in two different sentences, gets two different embedding vectors."

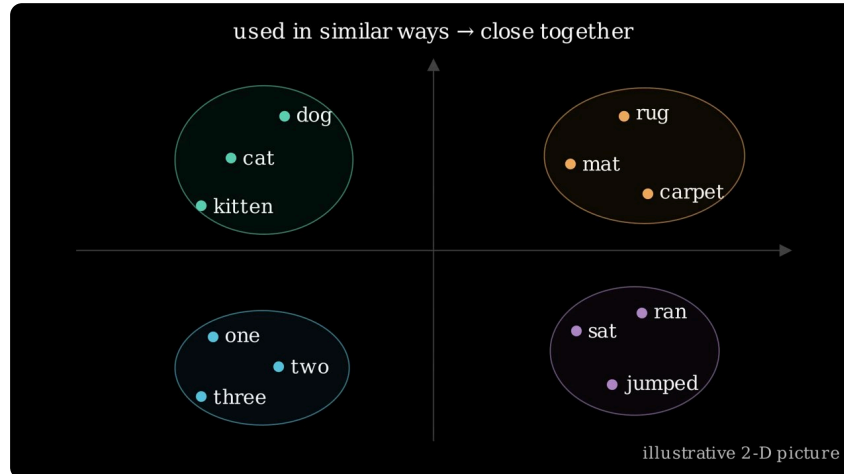
True ☐ False ☐

CONCEPT 3 OF 7

Similar tokens end up close together

▶ Video 0:58-1:12

Here's the key idea. After training, tokens that are **used in similar ways** end up **close together**. *cat* lands near *dog* and *kitten*; *mat* near *rug* and *carpet*. Numbers gather in one place, verbs in another.



▶ 1:11 After training: animals, floor coverings, numbers and verbs each gather in their own region (illustrative 2-D picture).

KEY IDEA

In embedding space, **closeness means similarity**: tokens used in similar ways get nearby points. Position reflects how a token is used, not how it is spelled.

Check 3: Similar tokens end up close together

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

The word *puppy* is added and the model is trained. In the figure's picture, which group would you expect it to join?

- A. cat, dog, kitten
- B. mat, rug, carpet
- C. one, two, three
- D. sat, ran, jumped

3.2 SHORT ANSWER

rug and *carpet* share no letters. Why do they still end up close together?

.....

3.3 TRUE OR FALSE

"After training, *sat* is closer to *ran* than to *rug*."

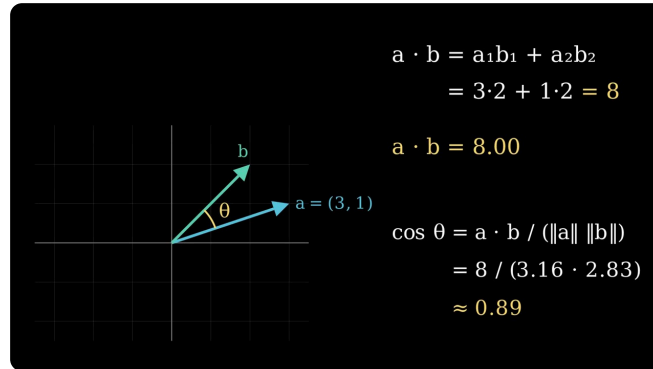
True ☐ False ☐

CONCEPT 4 OF 7

Measuring closeness: dot product and cosine

▶ Video 1:12-1:31

To measure “close”, use the **dot product**: multiply matching coordinates, then add them up. For $a = (3, 1)$ and $b = (2, 2)$: $a \cdot b = 3 \cdot 2 + 1 \cdot 2 = 8$. Vectors pointing the **same way** score high, unrelated (perpendicular) ones score **near zero**, and opposite ones go **negative**.



▶ 1:30 $a \cdot b = 3 \cdot 2 + 1 \cdot 2 = 8$, and $\cos \theta \approx 0.89$.

The dot product also grows with length. **Divide by both lengths** ($\|a\| = \sqrt{3^2 + 1^2} \approx 3.16$, $\|b\| \approx 2.83$) to get the **cosine similarity**, the cosine of the angle θ : $8 / (3.16 \cdot 2.83) \approx 0.89$. It runs from 1 (same direction) to -1 (opposite).

KEY IDEA

$a \cdot b = a_1b_1 + a_2b_2 + \dots$ High = same direction, ≈ 0 = unrelated, negative = opposite. **cos $\theta = a \cdot b / (\|a\| \|b\|)$** ignores length and measures direction only.

Check 4: Measuring closeness: dot product and cosine

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

Compute the dot product of $(2, -1, 3)$ and $(1, 4, 2)$.

4.2 MULTIPLE CHOICE

Which pair has a **negative** dot product?

- A. $(1, 0)$ and $(0, 1)$
- B. $(2, 1)$ and $(1, 2)$
- C. $(3, 1)$ and $(-3, -1)$
- D. $(1, 1)$ and $(5, 5)$

4.3 CALCULATION

Compute the cosine similarity of $(3, 4)$ and $(6, 8)$. What does the result tell you?

4.4 TRUE OR FALSE

“ $a = (3, 1)$ scores 10 with $(0, 10)$ but only 8 with $b = (2, 2)$, so $(0, 10)$ points more nearly the same way as a .”

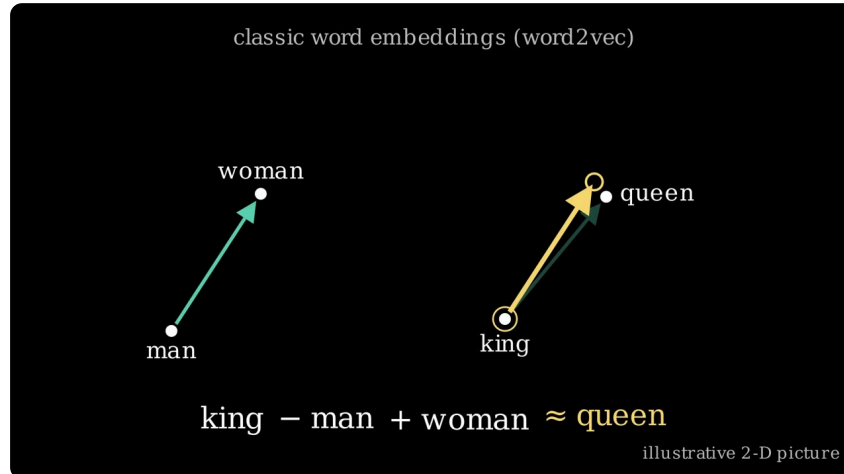
True ☐ False ☐

CONCEPT 5 OF 7

Directions carry meaning

▶ Video 1:31-1:45

Directions can carry meaning too. In classic word embeddings like **word2vec**, the step from *man* to *woman* is roughly the same as the step from *king* to *queen*. So vector arithmetic works: $\text{king} - \text{man} + \text{woman}$ lands **close to queen**.



▶ 1:44 The step from man to woman, added to king, lands close to queen (illustrative 2-D picture).

KEY IDEA

A **direction**, the step from one vector to another, can stand for a relationship. Adding the *man* → *woman* step to *king* moves it to about *queen*.

Check 5: Directions carry meaning

Answer all 3 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

Toy 2-D embeddings: $\text{man} = (1, 1)$, $\text{woman} = (1, 3)$, $\text{king} = (5, 1)$. Compute $\text{king} - \text{man} + \text{woman}$.

5.2 SHORT ANSWER

If the step from *France* to *Paris* means “capital of”, which word would you expect near $\text{Paris} - \text{France} + \text{Italy}$?

5.3 TRUE OR FALSE

“ $\text{king} - \text{man} + \text{woman}$ gives exactly the vector of queen.”

True ☐ False ☐

CONCEPT 6 OF 7

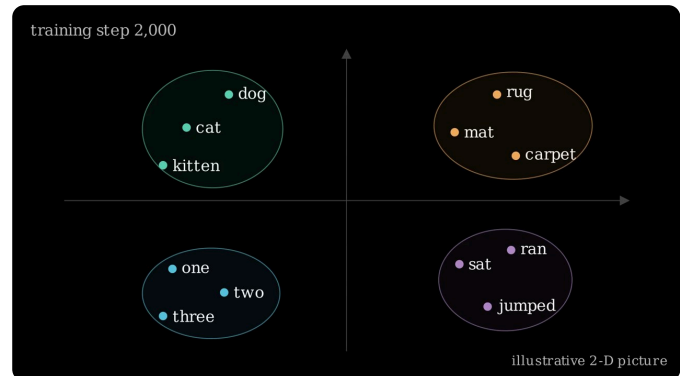
Learned, not designed

▶ Video 1:45–1:56

Nobody writes these numbers by hand. The embedding vectors **start out random**. Then **training nudges them**, a tiny bit at a time, until the geometry reflects how words are actually used. The clusters of concept 3 are a result of training, not a design.



▶ 1:48 Before training: the vectors are random, so the words are scattered.



▶ 1:55 After training: similar words have been nudged together.

KEY IDEA

The embedding matrix is part of the model's **parameters**: learned from data like the rest of its numbers (episode 1), starting from random values.

Check 6: Learned, not designed

Answer all 3 questions before looking at the answers. Pass mark: all correct.

6.1 TRUE OR FALSE

“Engineers place *cat* near *dog* by hand, using a dictionary.”

True ☐ False ☐

6.2 MULTIPLE CHOICE

Before training, where are *cat* and *dog* in the embedding space?

- A. Already next to each other
- B. At random places, unrelated to their meaning
- C. Both exactly at the origin
- D. At positions given by their IDs

6.3 SHORT ANSWER

The same model is trained twice, starting from different random numbers. Would *cat* get exactly the same vector both times? Would it still end up near *dog*?

CONCEPT 7 OF 7

Embeddings in code

▶ Video 1:56-2:25

In code, embedding is a single line. `E` has one row per token; indexing it with the list of IDs gives one vector per token, a 5×768 array for our 5-token sentence.

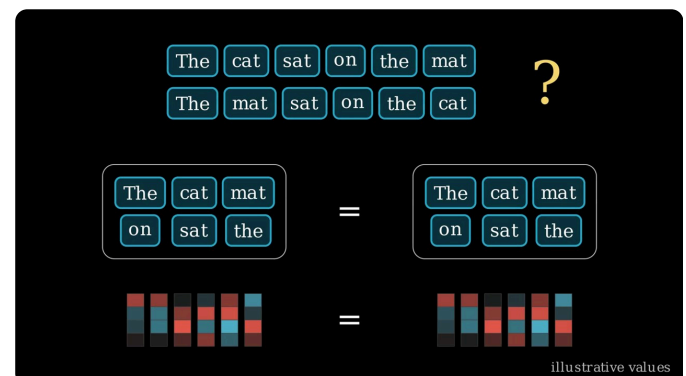
```
import numpy as np

vocab_size, d_model = 50257, 768
E = np.random.randn(vocab_size, d_model) * 0.02 # learned in training

ids = [464, 3797, 3332, 319, 262] # "The cat sat on the"
x = E[ids] # one row per token
print(x.shape) # (5, 768)
```

```
1 import numpy as np
2
3 vocab_size, d_model = 50257, 768
4 E = np.random.randn(vocab_size, d_model) * 0.02 # learned in training
5
6 ids = [464, 3797, 3332, 319, 262] # "The cat sat on the"
7 x = E[ids] # one row per token
8 print(x.shape) # (5, 768)
```

▶ 2:08 The whole embedding step: one line, `x = E[ids]`.



▶ 2:23 The catch: same tokens, same vectors. Nothing says which came first.

But there's a catch: *"The cat sat on the mat"* and *"The mat sat on the cat"* contain exactly the same tokens, so they get exactly the same vectors. Nothing says which came first. Next: position.

KEY IDEA

`x = E[ids]`: one row of `E` per token, shape (number of tokens, `d_model`). A token gets the same vector **wherever it is** in the sentence.

Check 7: Embeddings in code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

7.1 SHORT ANSWER

What does `print(x.shape)` show for *"The cat sat on the mat"* (6 tokens)? And for the original 5 tokens if `d_model` were 4,096?

7.2 MULTIPLE CHOICE

Why does the code fill `E` with small random numbers?

- A. It's a bug; real models use zeros
- B. It's only the starting point: training then adjusts the numbers
- C. Random vectors work as well as trained ones
- D. To hide the token IDs

7.3 HANDS-ON CODE

Try it yourself. The code below builds a toy E with 4 numbers per token and embeds the two sentences from the video. (a) Write `cosine(u, v)` (hint: `np.dot`, `np.linalg.norm`) and check it on (3, 1) and (2, 2). (b) Is `x1[1]` equal to `E[3797]`? What does `np.array_equal(x1, x2)` print, and why? (c) What does `np.allclose(x1.sum(axis=0), x2.sum(axis=0))` print? What does that mean for a model that only looks at the collection of vectors?

```
import numpy as np

def cosine(u, v):
    ... # (a) your code here

rng = np.random.default_rng(0)
E = rng.normal(size=(50257, 4)) # toy: 4 numbers per token
ids1 = [464, 3797, 3332, 319, 262, 2603] # "The cat sat on the mat"
ids2 = [464, 2603, 3332, 319, 262, 3797] # "The mat sat on the cat"
x1, x2 = E[ids1], E[ids2]
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. IDs are labels, vectors are points · rewatch ▶ 0:08-0:30

- 1.1 **False.** IDs are just labels; neighbouring numbers are unrelated tokens.
- 1.2 **B.** The vector works like coordinates, so every token becomes a point.
- 1.3 **B.** The size and closeness of IDs mean nothing, so arithmetic on them would be misleading.

2. The embedding matrix: one row per token · rewatch ▶ 0:30-0:58

- 2.1 **38,597,376 (about 38.6 million).** $50,257 \times 768$: one number per token per dimension.
- 2.2 **B.** Embedding is just a lookup: ID 319 grabs row 319.
- 2.3 **1,000 × 64.** One row per token, one column per number in each vector.
- 2.4 **False.** It's a lookup: the same ID always grabs the same row. This is the catch at the end of the video, and the topic of episode 4.

3. Similar tokens end up close together · rewatch ▶ 0:58-1:12

- 3.1 **A.** *puppy* is used much like *dog* and *kitten*.
- 3.2 **They are used in similar ways.** Position reflects usage, not spelling: *on the rug* and *on the carpet* appear in similar sentences.
- 3.3 **True.** *sat* and *ran* are both verbs and are used in similar ways; verbs gather in one region.

4. Measuring closeness: dot product and cosine · rewatch ▶ 1:12-1:31

- 4.1 **4.** $2 \cdot 1 + (-1) \cdot 4 + 3 \cdot 2 = 2 - 4 + 6 = 4$. Same rule in any number of dimensions.
- 4.2 **C.** They point in opposite directions: $-9 - 1 = -10$. A is 0 (perpendicular), B is 4, D is 10.
- 4.3 **1: same direction.** $50 / (5 \cdot 10) = 1$. (6, 8) is just (3, 4) stretched; cosine ignores length.
- 4.4 **False.** (0, 10) is just long. Its cosine with a is $10 / (3.16 \cdot 10) \approx 0.32$, far below 0.89. That's why we divide by the lengths.

5. Directions carry meaning · rewatch ▶ 1:31-1:45

- 5.1 **(5, 3).** The step man → woman is (0, 2); adding it to king gives (5, 3), where queen would be.
- 5.2 **Rome.** Paris – France is the “capital of” step; adding it to Italy lands near Italy's capital.
- 5.3 **False.** It lands close to queen, not exactly on it: the two steps are only roughly the same.

6. Learned, not designed · rewatch ▶ 1:45-1:56

- 6.1 **False.** Nobody writes the numbers. Training moves the vectors until similar words are close.
- 6.2 **B.** The vectors start random; meaning only shows up after training.
- 6.3 **No; yes.** The exact numbers depend on the random start, but in both runs training pulls words that are used alike close together.

7. Embeddings in code · rewatch ► 1:56-2:25

7.1 (6, 768) and (5, 4096). Rows = number of tokens; columns = `d_model`.

7.2 B. Embeddings start random and are learned (concept 6), hence the comment `# learned in training`.

7.3 (a) ≈ 0.894 · (b) True; False · (c) True (a) `def cosine(u, v): return np.dot(u, v) / (np.linalg.norm(u) * np.linalg.norm(v))` gives $8 / (3.16 \cdot 2.83) \approx 0.894$, as in the video. (b) Row 1 of `x1` is the lookup of ID 3797 (`_cat`), so yes. The arrays are not equal because the rows come in a different order. (c) The sum of the vectors is identical: the two sentences contain exactly the same six vectors. Anything that ignores order can't tell them apart, which is why episode 4 adds position.